Pedidos e Catálogo

Evoluir sem reescrever, sem janela de indisponibilidade e sem quebrar quem já usa

90 dias
app, parceiros e 2º país

10x
tráfego projetado

99,9 %
disponibilidade mensal

500 ms
p95 na criação de pedido

Recomendação: três ondas de 30/60/90 dias. A decisão mais arriscada (criar pedido sem duplicar e sem quebrar o contrato atual) já está provada em código.

Quatro falhas que pioram com 10x e dois bloqueios para crescer

P1 — Alto

Retry sem chave de idempotência

Pedido e cobrança em dobro. Mais tráfego, mais timeout, mais retry

P2 — Alto

Evento fora da transação

Parceiro recebe evento de pedido que não existe, ou não recebe nada

P3 — Alto

N+1 síncrono no Catálogo

p95 cresce com o número de itens; Catálogo fora derruba Pedidos

P4 — Médio

Sem snapshot de preço

O pedido "muda" quando o Catálogo muda. Sem auditoria

P5 — Bloqueio

Sem versionamento de API

Abrir para app e parceiros arrisca quebrar o web atual

P6 — Bloqueio

Dado pessoal sem separação por país

Impede a entrada no segundo país (residência de dados)

Recomendação: evoluir o sistema atual em três ondas

30 dias

Parar de errar

- Idempotency-Key no /v1 (opcional) e no /v2
- Snapshot de preço e descrição no item
- Evento gravado na outbox, na mesma transação
- Observabilidade e SLOs; /v2 implantado, fechado

60 dias

Escalar e abrir

- Catálogo em lote, com cache e circuit breaker
- Relay da outbox e broker
- Gateway com OAuth2 e quotas
- App no /v2; webhooks com parceiros piloto

90 dias

Expandir

- País 2: coluna de país ou célula mínima, conforme o parecer jurídico
- Parceiros em produção, com quotas por plano
- Depreciação do /v1 anunciada (≥ 6 meses)

Como cada entrega vai para produção: atrás de feature flag, migração só aditiva, canário 5 → 25 → 100 % com critério de parada (p95 > 500 ms ou 5xx > 0,1 %). Rollback por flag em até 5 min.

Decisão crítica: criar pedido sem duplicar e sem quebrar ninguém

O que entra em 30 dias

Idempotency-Key

mesma chave não cria outro pedido

/v1 intacto, /v2 novo

os dois lado a lado, mesmo backend

Snapshot

preço e descrição gravados no item

Outbox

pedido, chave e evento no mesmo COMMIT

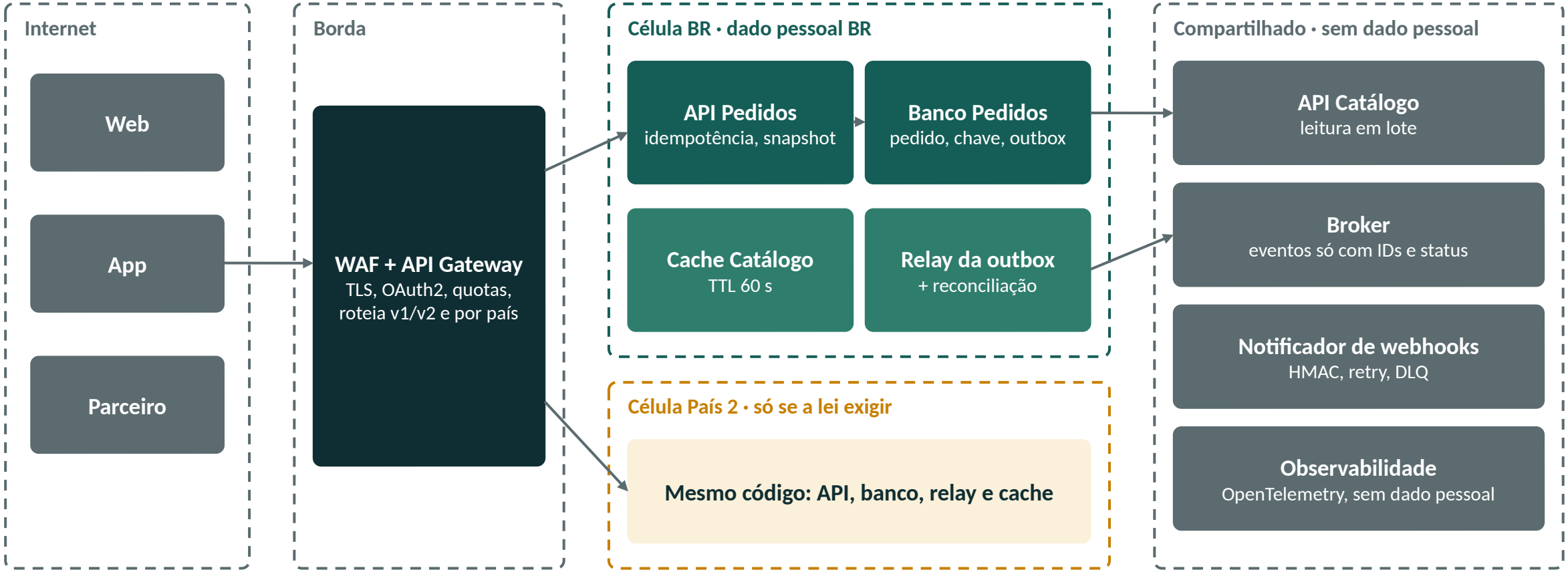
Quatro respostas possíveis na criação

201	Chave nova: grava pedido, chave e outbox numa transação
201 replay	Mesma chave e mesmo payload: devolve o pedido original
422	Mesma chave com outro payload: recusa, nada é gravado
409	Corrida rara: o vencedor ainda não está visível; o cliente repete

Duas chamadas simultâneas com a mesma chave: a UNIQUE barra a segunda, que relê e devolve o pedido original.

Por que começar aqui: ataca P1, P2, P4 e P5 sem infraestrutura nova e é pré-requisito do resto. Sem idempotência, todo retry futuro aumenta a duplicidade.

Arquitetura-alvo: o mesmo sistema, com fronteiras claras

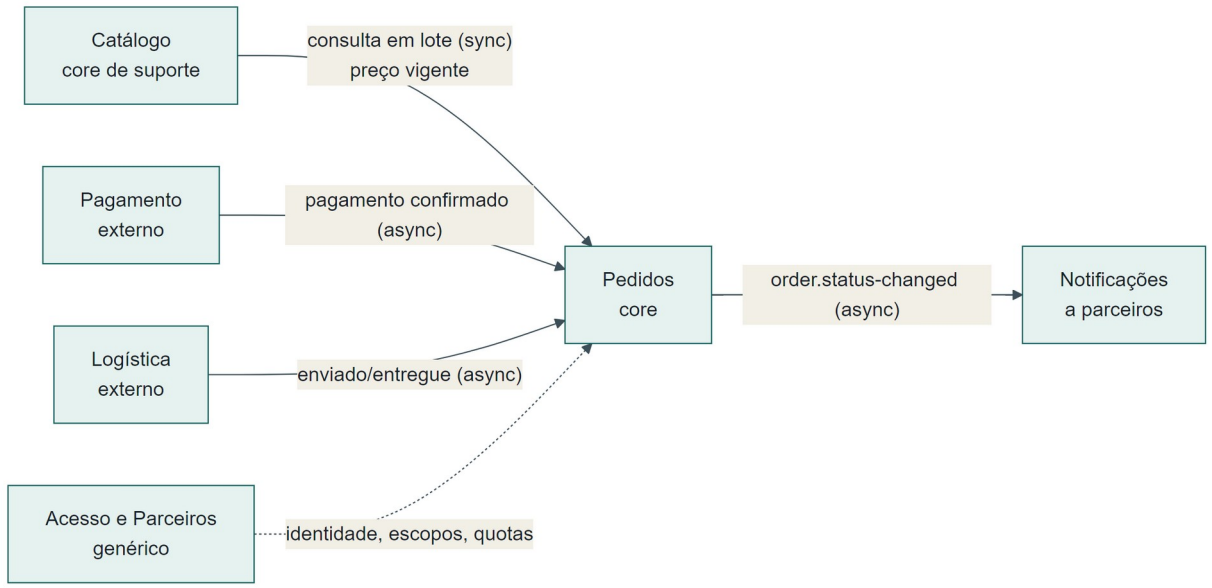


Dado pessoal não sai do país
Só Pedidos tem dado pessoal. Eventos, logs e Catálogo levam IDs.

Broker fora não para vendas
A outbox acumula; a criação de pedido segue.

p95 cabe com folga
Pior caso 300 ms (Catálogo em lote, sem cache) contra a meta de 500 ms.

Domínio: cada contexto é dono dos seus dados



Pedidos (core) possui pedido, itens com snapshot, status, chave e outbox. Não possui preço vigente nem cadastro do cliente.

Catálogo é upstream: Pedidos copia o preço no momento da compra.

Pagamento e Logística entram por camada anticorrupção.

Regra de consistência

Forte só dentro do banco de Pedidos de um país. Fora dele, eventual, sempre com janela definida e um mecanismo de correção.

Fluxo	Janela	Mecanismo
Pedido + chave + outbox	Forte (ACID)	mesma transação
Evento interno	≤ 5 min	relay + reconciliação
Webhook no parceiro	até 24 h	retry, DLQ, GET de reconciliação
Preço do snapshot	≤ 60 s	cache; catalogVersion no item

Segurança e LGPD: o dado pessoal fica onde precisa ficar

25

ameaças mapeadas por STRIDE, em cada fronteira de confiança, com controle e prazo

2

já provadas em teste: acesso a pedido de outro cliente (BOLA) e país forjado

Identidade

OAuth2 client_credentials para parceiros, OIDC para web e app. Token de 15 min, escopos mínimos, segredos em cofre.

Autorização

Por recurso, na API: parceiro só vê o pedido que criou (404 para o alheio). customerId vem do token, não do corpo.

Proteção de dados

Só Pedidos guarda dado pessoal. Eventos, webhooks, logs e prompts de IA levam só IDs; mascaramento no coletor.

LGPD e residência

Papéis definidos (parceiro controlador, plataforma operadora). Exclusão por pseudonimização. País 2: coluna ou célula, pelo parecer.

A prova: um comando, 52 testes, sem dependência externa

O teste prova que...	Tipo
Mesma chave + mesmo payload → o pedido original, sem duplicar	positivo
Chamadas concorrentes com a mesma chave → um único pedido	positivo
Mesma chave + payload diferente → 422, nada gravado	negativo
Consumidor web atual cria e consulta no /v1 como antes	contrato
Evento gravado na outbox junto com o pedido; replay não gera evento	positivo
Cliente não lê pedido de outro; evento sem dado pessoal	segurança

```
dotnet test
```

No CI (GitHub Actions)

- Spectral: lint das OpenAPI v1 e v2
- AsyncAPI CLI: valida o evento de status
- oasdiff: barra quebra de contrato (com prova negativa)
- NetArchTest: regras de camadas
- ADRs no formato e exceções vencidas

Uso de IA: prompts, validações e sugestões rejeitadas registrados em docs/uso-de-ia.md. Nada gerado entrou sem teste ou revisão; nenhum dado real ou segredo nos prompts.

Investimento da primeira fase e o que depende do cliente

~116 pessoa-dia

≈ R$ 93 mil de equipe em 30 dias (5,25 FTE)

≈ R$ 79 mil

com IA no desenvolvimento (−15 %); o prazo não muda

≈ R$ 3 a 7,5 mil

de infraestrutura na fase (ordem de grandeza)

Salários estimados por cargo; o custo real se fecha com os dados da empresa (folha, licenças, IA, tributos).

Riscos principais

- Tráfego real desconhecido: medir na semana 1
- Regra de residência do País 2 indefinida
- Consumidores atuais de eventos não mapeados
- Web demorar a enviar a chave no /v1

Decisões a validar com o cliente

- Baseline de tráfego e contrato v1 real
- País 2 e parecer jurídico até o dia 30
- Preço do momento do pedido; cache de 60 s
- Retenção da chave de idempotência em 24 h
- Nuvem, broker e gateway (critérios prontos)

Recomendação

30 **Parar de errar:** idempotência, snapshot e outbox, com o contrato atual intacto

60 **Escalar e abrir:** Catálogo em lote, broker, gateway e app no /v2

90 **Expandir:** segundo país (célula só se a lei exigir) e parceiros em produção

Sem reescrita, sem janela de indisponibilidade, e a decisão mais arriscada já está provada em código.

github.com/Derkoski/pedidos-catalogo · detalhes por critério no README